assignment2

October 21, 2026

1 CS 335 - Fall 2026

Assignment 2 Submission

Due (as printed in the source): Monday, Oct 26, 2025, 5:00pm.

The source's due-date year differs from its Fall 2026 heading; confirm the deadline with the course.

Name: Vlad Paun
Student number: 21291453

Instructor: Leili Rafiee Sevyeri
Email: Leili.rafiee.sevyeri@uwaterloo.ca

Submit all components of your solutions (written/analytical work, code/scripts/notebooks, figures, plots, etc.) to Crowdmark in PDF form in the section for each question.

You must also separately submit a single zip file containing any and all code/notebooks/scripts you write to the A2 DropBox on LEARN, in runnable format (that is, .ipynb).

For full marks, you must show your work and adequately explain your logic!

1.1 Question 1 - (15 marks) Option pricing using a binomial lattice.

Coding: Develop Jupyter/Python code for pricing European and American options, of both put and call type, using a binomial lattice. Use the lattice parameters for the no-arbitrage lattice, as described for example in Section 3.2 of the course notes (and in lectures). For this question, you do not need to apply smoothing of the payoff.

Your code should be vectorized so that there is only one explicit for loop over the timesteps (i.e., no for loops over vector entries within one timestep). See Section 3.5 of the course notes for hints on how to proceed. Also, your code should take only $O(N)$ storage, **NOT** $O(N^2)$. That is, do not store data for the whole tree at once, but rather data for one or two columns at a time. In the American case, ensure that you compute the payoff at each time step using vector operations; this will require determining/knowing the vector of stock prices for that time/column.

Test your code for European put/call options code by using the data in Table 1 below. Compare with the exact analytical solution values given by the function `blsprice` provided with the assignment. (`blsprice` is designed to mimic the Matlab function of the same name, so you can find details on it here: https://www.mathworks.com/help/finance/blsprice.html)

Generate text tables showing the option value $V_{\Delta t}(S_0^0, 0)$ (for both puts and calls) as a function of Δt. Start off with 500 timesteps (i.e., $\Delta t = T/500$), and then repeat the process doubling the

number of timesteps, i.e., 1000, 2000, ... 8000, timesteps. You should expect to see your results converging towards the exact solution value. Your tables be structured roughly like Table 2 (but you do not need to make fancy borders - just rows of text is fine). Your table should also show the exact (`blsprice`) value for comparison (where available).

Table 1 gives some typical option parameters.

Parameter	Value
σ	0.32
r	0.04
Time to expiry	1 year
Strike price K	\$100
Initial asset price S_0^0	\$100

If V^{exact} is the exact price, and V^{tree} is the price from a lattice pricer, then it is possible to prove that

$$V_{\Delta t}^{tree}(S,t) = V^{exact}(S,t) + C(S,t)\Delta t + O((\Delta t)^2), \tag{1}$$

Table 2 shows the convergence test layout.

Δt	Value	Change	Ratio
$T/500$	$V_{\Delta t_1}$		
$T/1000$	$V_{\Delta t_2}$	$V_{\Delta t_2} - V_{\Delta t_1}$	
$T/2000$	$V_{\Delta t_3}$	$V_{\Delta t_3} - V_{\Delta t_2}$	$\frac{V_{\Delta t_2} - V_{\Delta t_1}}{V_{\Delta t_3} - V_{\Delta t_2}}$
$\vdots$	$\vdots$	$\vdots$	$\vdots$
exact	V^{exact}		

where $C(S,t)$ is independent of Δt. Estimate (by hand calculation using the above formula)

$$\lim_{\Delta t \to 0} \frac{V_{\Delta t/2}^{tree}(S_0^0,0) - V_{\Delta t}^{tree}(S_0^0,0)}{V_{\Delta t/4}^{tree}(S_0^0,0) - V_{\Delta t/2}^{tree}(S_0^0,0)}. \tag{2}$$

What order of accuracy does the convergence data table you generated indicate our lattice method is likely exhibiting? Does this agree with the theory above?

Next, repeat the above tests for an **American put** (no analytic solution available), using the data in Table 1. Show a convergence table as in Table 2 (but without the exact solution).

1.1.1 Answer - code and results

[7]:
```
import numpy as np
from scipy.stats import norm
import matplotlib.pyplot as plt

#adapted from Octave's financial toolkit
def blsprice(Price, Strike, Rate, Time, Volatility):
    sigma_sqrtT = Volatility * np.sqrt (Time)

    d1 = 1 / sigma_sqrtT * (np.log(Price / Strike) + (Rate + Volatility**2 / 2)␣
 ↪* Time)
    d2 = d1 - sigma_sqrtT

    phi1 = norm.cdf(d1)
    phi2 = norm.cdf(d2)
    disc = np.exp (-Rate * Time)
    F    = Price * np.exp ((Rate) * Time)

    Call = disc * (F * phi1 - Strike * phi2)
    Put  = disc * (Strike * (1 - phi2) + F * (phi1 - 1))
    return Call, Put

#adapted from Octave's financial toolkit
def blsdelta(Price, Strike, Rate, Time, Volatility):
    d1 = 1 / (Volatility * np.sqrt(Time)) * (np.log (Price / Strike) + (Rate +␣
 ↪Volatility**2 / 2) * Time)

    phi = norm.cdf(d1)

    CallDelta = phi
    PutDelta = phi - 1
    return CallDelta, PutDelta

def lattice_pricer(N, Price, Strike, Rate, Time, Volatility, Call = True,␣
 ↪European = True):
    dt = Time / N

    a = np.exp(Rate * dt)
    discount = 1 / a

    u = np.exp(Volatility * np.sqrt(dt))
    d = 1 / u

    p = (a - d) / (u - d)
    q = 1 - p
```

```
    S = Price * np.exp((2 * (np.arange(N+1)) - N) * Volatility * np.sqrt(dt))

    if Call:
        V = np.maximum(S - Strike, 0)
    else:
        V = np.maximum(Strike - S, 0)

    for _ in range(0, N):
        V = discount * (p * V[1:] + q * V[:-1])

        if not European:
            S = S[1:] * d

            if Call:
                payoff = np.maximum(S - Strike, 0)
            else:
                payoff = np.maximum(Strike - S, 0)
            V = np.maximum(V,payoff)
    return V[0]
```

[8]:
```
def simulate(Timesteps, Price, Strike, Rate, Time, Volatility):

    Vs = np.zeros((len(Timesteps),2,2))
    for call, european in np.ndindex((2,2)):
        for i,N in enumerate(Timesteps):
            Vs[i,call,european] = lattice_pricer(N,Price, Strike, Rate, Time,␣
 ↪Volatility, call, european)
    return Vs

def construct_differences(Vs):
    return np.pad(
            Vs[1:] - Vs[:-1],
            ((1, 0), (0, 0), (0, 0))
        )

def construct_ratios(dVs):
    return np.pad(
        dVs[1:-1] / dVs[2:],
        ((2, 0), (0, 0), (0, 0))
    )
```

[21]:
```
import pandas as pd
from IPython.display import display

Volatility = 0.32
Rate = 0.04
Time = 1.0
```

```
Strike = 100.0
Price = 100.0
sims = 5
Timesteps = [500 * (2 ** i) for i in range(sims)]

Vs = simulate(Timesteps, Price, Strike, Rate, Time, Volatility)
dVs = construct_differences(Vs)
rVs = construct_ratios(dVs)

tables = {}

for call, european in np.ndindex((2, 2)):
    title = f"{'European' if european else 'American'} {'Call' if call else␣
 ↪'Put'}"

    df = pd.DataFrame({
        "Value": Vs[:, call, european],
        "Change": dVs[:, call, european],
        "Ratio": rVs[:, call, european]
    }, index=[f"{Time/N:.6f}" for N in Timesteps])

    if call or european:
        df.loc["exact"] = [
            blsprice(Price, Strike, Rate, Time, Volatility)[1-call],
            np.nan,
            np.nan
        ]

    tables[title] = df

table = pd.concat(tables, axis=1)

display(table.style.format("{:.6f}", na_rep=""))
```

```
<pandas.io.formats.style.Styler at 0x7f6cdfb3bed0>
```

1.1.2 Answer - hand calculation of the limit

Answer:

1.1.3 Answer - convergence order and comparison with theory

Answer:

[ ]:
```
# TODO: Generate the American put convergence table using Table 1.
# Include timestep size, value, change, and ratio.
```

1.1.4 Answer - American put observations

Answer:

1.2 Question 2 - (5 marks) Smoothing the Payoff.

In this question, we consider a variant of a put option known as a *digital put option*. Such an option is characterized by a strike price K and a payoff according to

$$\text{Payoff} = \begin{cases} 1 & \text{if } K > S, \\ 0 & \text{otherwise.} \end{cases}$$

where (as usual) S is the stock value. Determine the expressions for the *smoothed* digital put payoff in this case. You should use the same smoothing strategy as covered in the lecture (which will, of course, have to be modified for a digital put).

Answer:

1.3 Question 3 - (8 marks) Lattice properties.

Let C_j^n be the value of a European call option at node (j, n), obtained using a no-arbitrage lattice. Likewise, let P_j^n be the value of a European put option at node (j, n), obtained using a no-arbitrage lattice. Assume that P_j^n and C_j^n both have the same strike price K, are based on the same underlying stock S, and have expiry time $T = N\Delta t$. Show that

$$C_j^n - P_j^n + Ke^{-(N-n)r\Delta t} = S_j^n. \tag{3}$$

This is a discrete version of what is called *put-call parity*.

Hint: Induction is often useful for understanding lattices, and that is the case here as well. Work backwards from the expiry time (i.e., column N in the lattice) and use the rules that define the relationships between S at adjacent timesteps, and likewise for the option price V at adjacent timesteps (in this case we have used C or P rather than V to distinguish the two types of options, but the update rule remains the same).

Answer:

1.4 Question 4 - (8 marks) Dynamic Programming.

Consider the following game. You are given a fair **five-sided** die, with faces $\{0, 1, 2, 3, 4\}$. You can roll the die, and obtain a payoff equal to the value shown on the die, or you can choose to roll again. Note that if you choose to roll again, you can only get the new payoff or roll again. You can roll the die up to three times. You must pay \$1.4 for the first roll, \$1.05 for a second roll, and \$0.85. for a third roll. You pay only for the rolls you choose to take, and get no money back if you move onto the next roll. What is your expected gain, assuming you were to behave optimally at each step? (Hint: this is somewhat similar to the marriage problem we looked at in lecture. Use dynamic programming; i.e., work backwards from the last roll!)

Work this out manually, i.e., you do not need to write or run any code here.

Answer: